ShiftLink 운영 온톨로지와 상황 인식 실험 계획

Oct 22, 2026 · @재영

목표 흐름: MES 시뮬레이터가 이상 신호를 만든다 → 실시간 값으로 상황을 자동 인식한다 → 설비 관계를 따라 원인 후보와 지식을 찾는다 → 알림과 추천 조치를 표시한다.

결론: 합성 MES 안에서는 이 흐름이 동작한다. 온톨로지 v0와 지식 그래프, 판정기 시제품으로 PDP 전류 예제를 끝까지 흘렸다. 남은 일의 대부분은 알고리즘이 아니라 **지식 연결 채우기**와 **순환 없는 평가**다. 실제 설비 진단 정확도는 주장하지 않는다(임계값이 합성 값이다).

1. 진단: 지식 카드 스키마는 온톨로지가 아니다

카드는 문서 단위 레코드 스키마(`KnowledgeCard`)다. 신호·증상·원인·조치가 공유 ID가 아니라 카드 안의 문장으로 들어 있다.

확인 항목	현재	영향
MES 신호 조건	T2 14/86장. `<signal>_state` 관례는 어디에도 선언되지 않음. 2장은 MES로 판정 불가	72장은 MES 값과 기계적으로 연결되지 않음
증상·원인·조치	자유 문장. T3 조치 단계 108개만 구조화(순서·전제·멈춤 조건)	조치 추천 재료는 T3 단계뿐
부품	자유 문자열("주 베인펌프"), 카탈로그 부품 ID와 연결 없음	고장 모드와 이어지지 않음
설비 범위	`mes_equipment_id` 44/86장	나머지는 유형 단위

따로 존재하지만 서로 ID로 이어지지 않는 어휘가 셋 있다.

1. **카탈로그:** 설비 간 관계 20개(전력·유압·공압 공급, 구동, 소재 흐름, 인터록, 인과 여부, 지연), 부품 18, 고장 모드 41. 사실상 공장 도메인 모델.
2. **MES 증상 패턴(`symptoms.py`):** 신호 상태 조합 15개 → 증상·원인 후보·확인 항목. 같은 표로 무작위 고장도 주입한다.
3. **카드와 kg-demo-v1:** 카드 86, 문서 판본 92, 증상 개념 18. 발표용이며 검색에 쓰지 않는다. 증상 개념 중 센서로 관측 가능한 것은 약 10개.

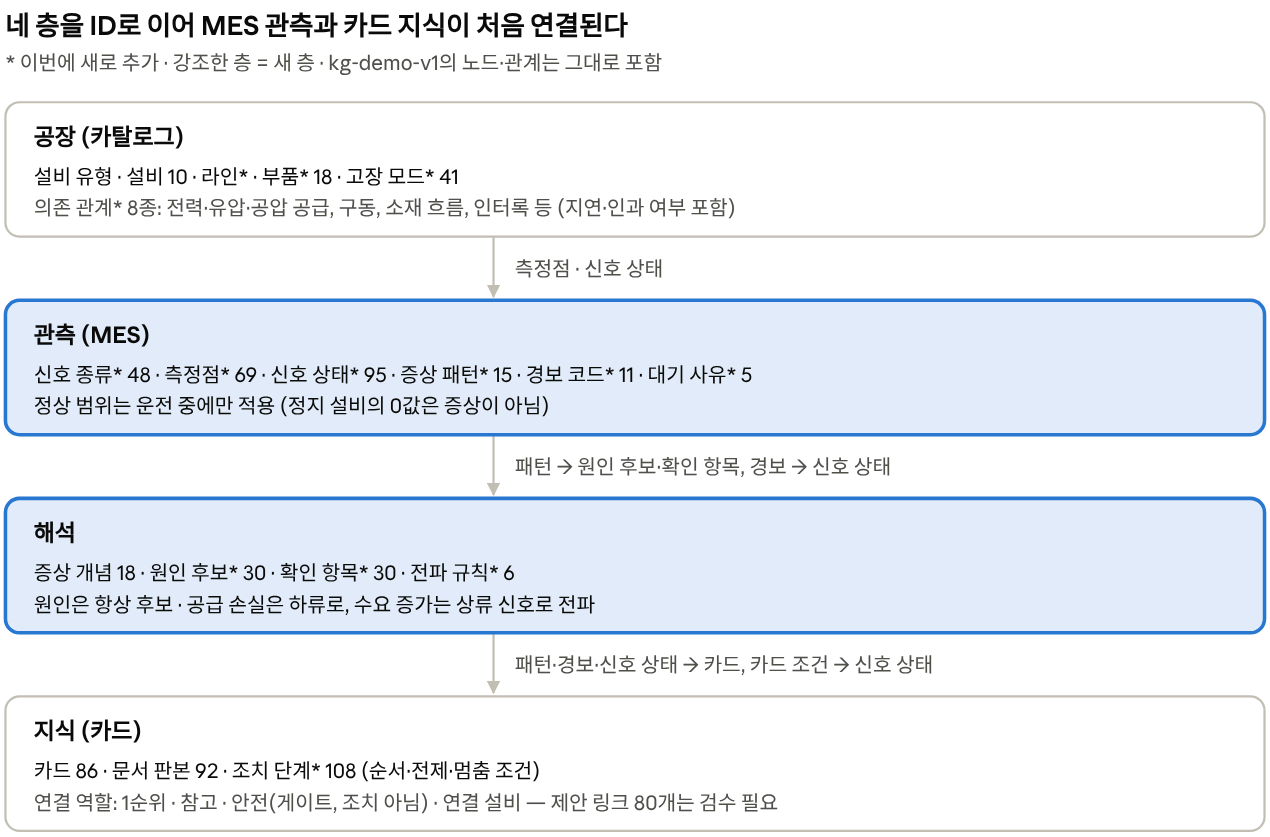
첨부: 부록 A `coverage.py` — 패턴·신호 상태별 카드 연결 범위 측정

2. 온톨로지 설계 (v0 제안)

원칙

1. 유형 지식과 개체 관측을 나눈다(신호 종류 vs 설비별 측정점).
2. 관측과 해석을 나눈다. 원인은 항상 "후보"다.
3. 정상 범위는 운전 중에만 뜻이 있다. 정지 설비의 0값은 증상이 아니다.
4. 관계는 방향과 인과 여부를 가진다. 공급 손실은 하류로, 수요 증가는 상류 신호로 전파된다. 동시 발생(co_occurrence) 관계는 원인 추론에 쓰지 않는다.
5. 안전 카드는 게이트이지 조치가 아니다.
6. 제안·추론 링크에는 검수 상태를 단다.

층	클래스 (새로 더한 것 굵게)	출처	수
공장	설비 유형, 설비, **라인, 부품, 고장 모드**	카탈로그	83, 10, 1, 18, 41
관측	**신호 종류, 측정점, 신호 상태, 증상 패턴, 경보 코드, 대기 사유**	MES 구성·symptoms.py	48, 69, 95, 15, 11, 5
해석	증상 개념, **원인 후보, 확인 항목, 전파 규칙**	kg-demo-v1·symptoms.py·제안	18, 30, 30, 6
지식	카드, 문서 판본, **조치 단계**	kg-demo-v1·카드	86, 92, 108



운영 온톨로지 v0 · 4개 층

관측과 해석 층이 새로 생기면서, MES 신호 상태에서 원인 후보와 카드까지 ID로 따라갈 수 있게 됐다.

주요 관계: 설비 의존 8종(공급자 → 의존 설비, 인과·지연 포함), 패턴 → 필요한 신호 상태, 패턴 → 원인 후보·확인 항목, 경보 → 알리는 신호 상태, 카드 → 적용 조건·적용 설비·조치 단계, 패턴·경보·신호 상태 → 카드(역할: 1순위·참고·안전·연결 설비).

그래프 v0: 노드 765, 관계 1,153. kg-demo-v1의 노드 289·관계 434는 바꾸지 않고 포함했다. 패턴·경보 → 카드 링크 73개와 증상 연결 7개, 전파 규칙 6개는 **제안 상태라 사람 검수가 필요**하다.

첨부: 부록 B `build_kg.py` (그래프 빌더, 제안 링크 표 포함) · `validation_v0.json` (집계와 검증)

3. 흐름 시험 결과

판정기는 실제 MES가 줄 수 있는 정보만 읽는다(최근 3프레임 측정값, 증상 판정, 경보 코드, 운전 상태·대기 사유). 시나리오 ID와 고장 정답은 읽지 않는다. PC 기준 매 틱 2.8 ms.



MES 이상 → 알림 흐름 · 6단계

①–③은 MES에 이미 있고, 이번에 ④ 관계 추론과 ⑤ 지식 연결을 만들었다. ⑥ PDA 알림은 실험 E4에서 만든다.

예: PDP-01 배전반 전류 50 A (기준 20–40)

- 상황: 버스 전류 상승, 트립 전 관측 (자동 판정 · 확정 아님)
- 원인 후보: 부하 증가 또는 기동 전류 / 과전류 원인 가능성
- 관계: GR-01·GR-02·HPU-01(전력 공급) 부하 신호 정상 → 하류 부하 증가 근거 없음
- 확인할 것: 기동·부하 이력과 차단기 정격·설정 기록 / 3상 전류 파형과 보호계전 이벤트
- 참고 카드 K-1317·K-1325, 안전 K-1309(충전부 접촉 금지)
- **지식 공백:** 직접 쓰는 카드 없음 → 조치 후 근무일지 기록 시 카드 후보로 검토

GR-01 부하 증가를 함께 주입하면 PDP 알림이 "GR-01 구동 전류 높음 → 하류 부하 증가 후보"로 원인 쪽 설비를 짚는다. 유압 고장·감속기 과열에서는 멈춘 하류 설비를 따로 띄우지 않고 "영향: RT-01 대기"로 원인 설비 알림에 묶는다.

패턴 표 밖의 시나리오	현재 MES 증상 판정	판정기	엉뚱한 설비 지목	조치 카드 연결
이름 붙은 고장 11	2/11	11/11	0	10/11
신호 하나 이상 69	3/69	69/69	0	7/69

첨부: 부록 C `situation.py` (상황 판정기와 데모) · `eval_scenarios.py` (시나리오 기준값 측정)

4. 위험

1. **순환:** 이상 생성기와 판정기가 같은 패턴 표를 쓴다. 무작위 고장 317건 93%·오탐 0은 구조상 당연한 결과다. 위 표도 경보가 원인 설비에 뜨고 링크를 같은 사람이 만든 쉬운 조건이다.
2. **지식 공백이 가장 큰 병목:** 카드 연결은 패턴 6/15, 신호 상태 7/96. 대부분의 알림이 공백으로 끝나며, 근무일지 → 카드 게시 흐름으로 메울 대상이다.
3. **고친 오류:** 정지 설비의 0값을 이상으로 판정, 상황끼리 서로를 원인으로 지목하는 모순. v0에서 운전 문맥과 전파 규칙으로 수정했다.
4. **범위:** 합성 MES에서의 흐름 검증까지만 주장한다.

첨부: 부록 D `probe_screen.py` (무작위 고장 317건 순환 확인) · `probe_named.py` (패턴 표 밖 시나리오 확인)

5. 실험 계획 (약 3일)

가설	핵심 지표	성공 기준
H1 상황 인식	경보를 가린 상태의 원인 설비 인식률, 오알림	정상 강도 ⩾ 85%, 약한 고장 ⩾ 60%, 오알림 ⩽ 1회/실행 시간, p95 ⩽ 5틱
H2 원인 위치	대기 사유를 가린 상태의 근본 설비 1순위 적중, 상황 간 모순	⩾ 80%, 모순 0, 잘못 지목 ⩽ 10%
H3 지식 연결	독립 정답 대비 상위 3개 카드 적중, 공백 판정, 안전 위반	⩾ 80%, 공백 F1 ⩾ 0.9, 안전 위반 0
H4 알림 품질	블라인드 사람 평가(정확·안전·유용)	⩾ 4.0/5, 위험 판정 0
H5 Jetson	틱당 추가 지연, C5 질의 p95, 여유 메모리	⩽ 50 ms, p95 변화 ⩽ 0.5초, ⩾ 1.0 GiB

데이터: 무작위 고장(위생 점검만), 약화·간헐 변형, 이름 붙은 고장 11과 신호 이상 69(경보 가린 변형 포함), 정상 운전, 동시·전파 고장 스크립트, 가능하면 팀원 v7 생성기(링크 전파). 개발 seed로만 반복하고 시험 seed는 동결 뒤 한 번만 돌린다.

정답: 상황 유형마다 1순위·참고·안전 카드 또는 "없음"을 사람이 작성한다. 제안 링크를 보지 않고 작성하며, 20%는 두 명이 겹쳐 일치도를 잰다.

단계	내용	기간
E0	온톨로지·그래프·판정기 시제품·기준값	완료
E1	제안 링크·규칙 검수, 정답 독립 작성, 사전 등록 동결	0.5일 (사람 필요)
E2	평가 하네스(재생·채점·가림 변형), 개발 seed로 반복	1일
E3	시험 1회, 결과·실패 분류	0.5일
E4	PDA 알림 시제품(기본 꺼짐), Jetson 지연, 블라인드 평가	0.5–1일

실패 분류: 센서로 관측 불가 / 패턴 미정의 / 운전 문맥 오판 / 관계 규칙 오적용 / 카드 링크 누락 / 진짜 지식 공백 / 정답 모호 / 알림 폭주 / 안전 게이트 위반.

안전 정책: 추천 조치는 확인 절차로만, 멈춤 조건을 함께 표시한다. 안전 카드는 게이트로만 쓴다. 알림에 "자동 판정 · 확정 아님"을 단다. 장비 제어는 하지 않는다.

협의할 것: kg-demo-v1 계약과 MES 증상 패턴은 팀원 소관이므로 확장 제안으로 공유하고, 병합은 합의 뒤에 한다. MES 탐지 연구(EWMA)는 나중에 증거 유형으로, 지식 공백은 근무일지 → 카드 게시 흐름과 연결한다.

자료: ShiftLink-records/experiments/ontology_20261022 (ONTOLOGY.md, PLAN.md, build_kg.py, out/graph_v0.json, situation.py, 측정 스크립트)

첨부: 부록 E `PLAN.md` (실험 계획 전문)

부록: 첨부 파일

파일 원본은 PC의 `ShiftLink-records/experiments/ontology_20261022/` 에 있다. 실행은 저장소 루트를 첫 인자로 준다(예: `python -X utf8 build_kg.py <ShiftLink 저장소> out`). `graph_v0.json` (914 KB)은 문서에 넣기에 커서, `build_kg.py` 로 같은 파일을 만들 수 있다.

부록 A. coverage.py

```
"""Coverage (10/22): how much of what the MES can detect already links to
knowledge cards?
Situations: 13 random-factory patterns (+pdp_trip, rt_lift), single-signal
low/high states of every MES signal, named
scenario alarm codes. Links: exact = a card condition (<signal>_state /
signal) satisfied by the situation;
text = an L1 card of the same equipment type whose title/symptom names the
signal (candidate for human review only).
  python -X utf8 coverage.py REPO"""
import collections, json, re, sys
from pathlib import Path
repo = Path(sys.argv[1]); sys.path.insert(0, str(repo))
from shiftlink.mes.configuration import from_catalog
from shiftlink.mes.symptoms import PATTERNS
cat = json.loads((repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))
cfg = from_catalog(cat)
cards = json.loads((repo /
"docs/data/knowledge_cards/kb/kb_cards.json").read_text(encoding="utf-8"))
["cards"]
names = {}
for e in cfg.equipment:
    for s in e.signals: names[s.signal] = (s.name.split(" (")[0],
e.code.split("-")[0])

def exact(states):   # states: {signal: low/high/normal/zero}
    out = []
    for c in cards:
        cond = c.get("conditions") or []
        if not cond: continue
        ok = True
        for k in cond:
            sig = k["signal"]; base = sig[:-6] if sig.endswith("_state") else
sig
            if base not in states: ok = False; break
            st = states[base]
            val = st if sig.endswith("_state") else (1 if st == "high" else 0
if st in ("zero", "normal") else None)
            if val is None or not (val == k["value"] if k["op"] == "==" else
False): ok = False; break
```

```
        if ok: out.append(c["card_id"])
    return out

def text(sig):
    nm, typ = names[sig]
    key = re.sub(r"\s+", "", nm)
    return [c["card_id"] for c in cards if c["equipment"] == typ and key and
key in re.sub(r"\s+", "", c["title"] + (c.get("symptom") or ""))]

print("== patterns (multi-signal)")
pat_exact = 0
for p in PATTERNS:
    ex = exact(p.states); tx = sorted({x for s, st in p.states.items() if st
!= "normal" for x in text(s)})
    pat_exact += bool(ex)
    print(f"  {p.pattern_id:16s} exact={ex} text={tx[:6]}")
print("== single-signal states")
rows = []
for sig, (nm, typ) in sorted(names.items()):
    for st in ("low", "high"):
        rows.append((sig, st, exact({sig: st}), text(sig)))
n = len(rows); ex = sum(bool(r[2]) for r in rows); tx = sum(bool(r[3]) and
not r[2] for r in rows)
print(f"  {n} signal-states: exact card {ex}, text-candidate only {tx}, none
{n - ex - tx}")
print("  exact:", [(r[0], r[1], r[2]) for r in rows if r[2]])
print(f"patterns with an exact card: {pat_exact}/{len(PATTERNS)}")
```

부록 B. build_kg.py

```
"""ShiftLink operational knowledge graph, proposal v0 (10/22). Extends kg-
demo-v1 (docs/demo/kg/build/graph.json, read
only) with the layers the MES -> situation -> knowledge flow needs: signals
and their states, symptom patterns of the
MES screen, plant dependencies, alarm types, T3 steps and card <-> MES links.
Same JSON shape and id formats as
kg-demo-v1; its nodes and edges are copied unchanged.

Sources (all read only): catalog 00_plant_and_relations.json, MES
configuration (from_catalog + priority scenarios),
shiftlink/mes/symptoms.py PATTERNS, kb_cards.json,
docs/demo/kg/build/graph.json.
```

```
Exact edges come from the sources. Edges in CURATED below are Claude's
proposals (review_status "proposed"), to be
reviewed by a person before any use beyond the experiment; the experiment's
gold mapping must not be made from them.

  python -X utf8 build_kg.py REPO OUT_DIR
"""
import hashlib
import json
import sys
from pathlib import Path

repo, out = Path(sys.argv[1]), Path(sys.argv[2])
sys.path.insert(0, str(repo))
from shiftlink.mes.configuration import from_catalog  # noqa: E402
from shiftlink.mes.engine import MesEngine  # noqa: E402
from shiftlink.mes.scenarios.priority import expand  # noqa: E402
from shiftlink.mes.symptoms import PATTERNS  # noqa: E402

cat = json.loads((repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))
cfg = expand(from_catalog(cat))
cards = json.loads((repo /
"docs/data/knowledge_cards/kb/kb_cards.json").read_text(encoding="utf-8"))
["cards"]
base = json.loads((repo /
"docs/demo/kg/build/graph.json").read_text(encoding="utf-8"))

# Proposed links (review_status "proposed"). role: primary = handles this
situation; partial = needs a person's
# observation or a narrower condition; safety = caution to show, never an
action; relation = card of the asset a
# dependency points at.
CURATED = {
    "pat:hpu_delivery": {"primary": ["K-1203"], "partial": ["K-1004"],
"safety": ["K-1006"]},
    "pat:hpu_restriction": {"primary": ["K-1201", "K-1202"], "partial": ["K-
1203"], "safety": ["K-1006"]},
    "pat:hpu_heat": {"primary": ["K-1002"], "safety": ["K-1006"]},
    "pat:cau_supply": {"primary": ["K-1312"], "partial": ["K-1301", "K-
1303"], "safety": ["K-1314"]},
    "pat:cau_flow_demand": {"primary": ["K-1301", "K-1303"], "partial": ["K-
```

```
1302"], "safety": ["K-1320"]},
    "pat:gr_lubrication": {"primary": ["K-1210", "K-1209"], "partial": ["K-
1025"], "safety": ["K-1024"]},
    "pat:gr_load": {"safety": ["K-1024"]},
    "pat:rt_resistance": {"primary": ["K-1208", "K-1207"], "partial": ["K-
1017"]},
    "pat:rt_clamp": {"relation": ["K-1203"]},
    "pat:rt_lift": {"partial": ["K-1022"], "safety": ["K-1018"]},
    "pat:cv_slip": {"primary": ["K-1204"], "partial": ["K-1205"], "safety":
["K-1014"]},
    "pat:cv_resistance": {"primary": ["K-1206"], "partial": ["K-1015"],
"safety": ["K-1014"]},
    "pat:pdp_voltage": {"primary": ["K-1307"], "partial": ["K-1310"],
"safety": ["K-1309"]},
    "pat:pdp_current": {"partial": ["K-1317", "K-1325"], "safety": ["K-
1309"]},
    "pat:pdp_trip": {"primary": ["K-1308", "K-1317"], "partial": ["K-1306",
"K-1310"], "safety": ["K-1309"]},
    "alarm:AL-HYD-HOT": {"primary": ["K-1002"], "safety": ["K-1006"]},
    "alarm:AL-HYD-LOW": {"primary": ["K-1203"], "partial": ["K-1001"],
"safety": ["K-1006"]},
    "alarm:AL-GR-HOT": {"primary": ["K-1210", "K-1025"], "partial": ["K-
1216"], "safety": ["K-1024"]},
    "alarm:AL-GR-LEAK": {"primary": ["K-1027"], "safety": ["K-1024"]},
    "alarm:AL-AIR-LOW": {"primary": ["K-1312"], "partial": ["K-1301"],
"safety": ["K-1314"]},
    "alarm:AL-PDP-TRIP": {"primary": ["K-1308", "K-1317"], "partial": ["K-
1306"], "safety": ["K-1309"]},
    "alarm:AL-DRV-VIB": {"primary": ["K-1209"], "safety": ["K-1024"]},
    "alarm:AL-DSB-QUE": {"primary": ["K-1206"], "partial": ["K-1215"],
"safety": ["K-1014"]},
    "st:hpu_pump_outlet_pressure=low": {"primary": ["K-1001"], "safety": ["K-
1006"]},
    "st:air_nozzle_pressure=high": {"safety": ["K-1321"]},
    "st:gr_oil_leak=high": {"primary": ["K-1027"]},
    "st:fluid_viscosity=high": {"primary": ["K-1007"]},
}
# pattern -> kg-demo-v1 symptom concept (proposed)
PATTERN_SYMPTOM = {
    "pat:hpu_delivery": ["sym:hpu-outlet-pressure-low-present", "sym:hpu-
pump-noise-low-output-milky-foam"],
    "pat:hpu_restriction": ["sym:hpu-filter-dp-above-1-2-bar"],
    "pat:hpu_heat": ["sym:hpu-cooler-outlet-oil-temp-over-limit"],
```

```
    "pat:cau_supply": ["sym:cau-air-pressure-drop"], "pat:cau_flow_demand":
["sym:cau-air-pressure-drop"],
    "pat:pdp_voltage": ["sym:pdp-bus-voltage-dip-at-motor-start"],
}
# How a disturbance travels along a causal dependency (provider ->
dependent). down: provider state -> dependent
# effect; up: a dependent's extra demand shows up in the provider's signal.
Used by the resolver; proposed.
PROPAGATION = [
    {"id": "prop:power-loss", "relation": "power_supply", "dir": "down",
"provider": ["breaker_trip=high", "bus_voltage=low"],
     "dependent": ["wait:power_supply_fault"]},
    {"id": "prop:power-demand", "relation": "power_supply", "dir": "up",
"provider": ["bus_current=high"],
     "dependent": ["gr_current=high", "hpu_pump_current=high"]},
    {"id": "prop:hydraulic-loss", "relation": "hydraulic_supply", "dir":
"down", "provider": ["hpu_pressure=low", "hpu_flow=low"],
     "dependent": ["rt_clamp_press=low", "rt_lift_delay=high",
"cv_belt_tension=low", "wait:hydraulic_supply_low"]},
    {"id": "prop:air-loss", "relation": "pneumatic_supply", "dir": "down",
"provider": ["air_pressure=low"],
     "dependent": ["wait:pneumatic_supply_low"]},
    {"id": "prop:drive-loss", "relation": "drive", "dir": "down", "provider":
["gr_rpm=low"],
     "dependent": ["rt_speed=low", "cv_speed=low",
"wait:upstream_drive_fault"]},
    {"id": "prop:block", "relation": "interlock", "dir": "down", "provider":
["cv_queue_len=high"],
     "dependent": ["rt_speed=low", "wait:downstream_block"]},
]
WAIT_RELATION = {"power_supply_fault": "power_supply",
"hydraulic_supply_low": "hydraulic_supply",
                 "pneumatic_supply_low": "pneumatic_supply",
"upstream_drive_fault": "drive", "downstream_block": "interlock"}
CAUSAL_TYPES = {"power_supply", "hydraulic_supply", "pneumatic_supply",
"drive", "material_flow", "interlock", "common_mode"}

nodes = {n["id"]: n for n in base["nodes"]}
edges = [dict(e) for e in base["edges"]]
asset_of = {n["props"]["equipment_id"]: n["id"] for n in base["nodes"] if
n["type"] == "Asset"}
card_node = {n["props"]["card_id"]: n["id"] for n in base["nodes"] if
n["type"] == "CardVersion"}
```

```
code_of = {e.equipment_id: e.code for e in cfg.equipment}
fam_of = {e.equipment_id: e.code.split("-")[0] for e in cfg.equipment}


def node(i, typ, label, **props):
    assert i not in nodes or nodes[i]["type"] == typ, i
    nodes.setdefault(i, {"id": i, "label": label, "type": typ, "props":
props})
    return i


seen = {(e["source"], e["target"], e["type"]) for e in edges}


def edge(s, t, typ, **props):   # one edge per (source, target, type): a
state shared by GR-01/GR-02 is added once
    if (s, t, typ) not in seen:
        seen.add((s, t, typ))
        edges.append({"source": s, "target": t, "type": typ, "props": props})


# plant: line, parts, failure modes, dependencies
line = node("line:LN-0001", "Line", "LN-0001", is_synthetic=True)
for c in cat["components"]:
    p = node(f"part:{c['component_id']}", "AssetPart", f"
{code_of[c['equipment_id']]} {c['name']}", code=c["code"],
             kind=c["component_kind"], is_synthetic=True)
    edge(asset_of[c["equipment_id"]], p, "HAS_PART",
basis="catalog_components")
    for k, fm in enumerate(c["failure_modes"]):
        edge(p, node(f"fm:{c['component_id']}/{k}", "FailureMode", fm,
component_id=c["component_id"]), "HAS_FAILURE_MODE")
for r in cat["relations"]:
    src = asset_of.get(r["from_id"])
    dst = asset_of.get(r["to_id"]) or (line if r["to_id"] == "LN-0001" else
None)
    if src and dst:
        edge(src, dst, r["relation_type"].upper(),
relation_id=r["relation_id"], causal=r["causality"] == "causal",
             medium=r["transfer_medium"], lag_seconds=r["lag_seconds"],
evidence_strength=r["evidence_strength"],
             note=r.get("note"))
```

```
# signals: kind (type level, where knowledge attaches) and point (asset
level, where the MES measures)
for e in cfg.equipment:
    for s in e.signals:
        k = node(f"sigk:{s.signal}", "SignalKind", s.name.split(" (")[0],
unit=s.unit)
        p = node(f"sig:{e.equipment_id}/{s.signal}", "SignalPoint", f"
{e.code} {s.name.split(' (')[0]}", unit=s.unit,
                 normal_min=s.normal_min, normal_max=s.normal_max,
acquisition=s.semantics.get("acquisition"),
                 location=s.semantics.get("location"))
        edge(asset_of[e.equipment_id], p, "HAS_SIGNAL"); edge(p, k,
"INSTANCE_OF")
        for st in (("low", "high") if s.unit != "bool" else ("high",)):
            edge(node(f"st:{s.signal}={st}", "SignalState", f"{s.name.split('
(')[0]} {'낮음' if st == 'low' else '높음'}",
                      signal=s.signal, state=st), k, "STATE_OF")
node("st:bus_current=zero", "SignalState", "배전반 전류 0",
signal="bus_current", state="zero")
edge("st:bus_current=zero", "sigk:bus_current", "STATE_OF")

# MES screen patterns: required states, causes, checks
for p in PATTERNS:
    pid = node(f"pat:{p.pattern_id}", "SymptomPattern", p.symptom,
family=p.family, limitation=p.limitation,
               sources=list(p.sources), basis="shiftlink/mes/symptoms.py")
    edge(pid, f"type:{p.family}", "APPLIES_TO")
    for sig, st in p.states.items():
        if st == "normal":
            edge(pid, f"sigk:{sig}", "REQUIRES_NORMAL")
        else:
            edge(pid, f"st:{sig}={st}", "REQUIRES")
    for k, c in enumerate(p.candidates):
        edge(pid, node(f"cause:{p.pattern_id}/{k}", "CauseHypothesis", c),
"SUGGESTS", rank=k)
    for k, c in enumerate(p.checks):
        edge(pid, node(f"chk:{p.pattern_id}/{k}", "Check", c),
"RECOMMENDS_CHECK", order=k)

# alarm types and wait reasons the MES raises; scenario effects tie an alarm
to the state it reports
for code, label in {**MesEngine.ALARM_LABELS, "AL-RANDOM-PERFORMANCE": "합성
성능 경고"}.items():
```

```
    node(f"alarm:{code}", "AlarmType", label, code=code)
spec_of = {e.equipment_id: {s.signal: s for s in e.signals} for e in
cfg.equipment}
for sc in cfg.scenarios:
    if not sc.alarm_code or sc.scenario_id.startswith(("sensor_anomaly_",
"symptom_")):
        continue
    node(f"alarm:{sc.alarm_code}", "AlarmType",
MesEngine.ALARM_LABELS.get(sc.alarm_code, sc.alarm_code), code=sc.alarm_code)
    for fx in sc.signal_effects:
        s = next((sp[fx.signal] for sp in spec_of.values() if fx.signal in
sp), None)
        if s and s.normal_min is not None and s.unit != "bool":
            st = "low" if fx.value < s.normal_min else "high" if fx.value >
s.normal_max else None
        else:
            st = "high" if s and fx.value >= 1 else None
        if st:
            edge(f"alarm:{sc.alarm_code}", f"st:{fx.signal}={st}", "REPORTS",
basis=f"scenario:{sc.scenario_id}")
for reason, rel in WAIT_RELATION.items():
    node(f"wait:{reason}", "WaitReason", reason, relation_type=rel)
for pr in PROPAGATION:
    pn = node(pr["id"], "PropagationRule", pr["id"][5:],
relation_type=pr["relation"], direction=pr["dir"],
              review_status="proposed")
    for s in pr["provider"]:
        edge(pn, f"st:{s}" if "=" in s else s, "PROVIDER_STATE")
    for s in pr["dependent"]:
        edge(pn, s if s.startswith("wait:") else f"st:{s}",
"DEPENDENT_STATE")

# cards: exact MES conditions, asset scope, T3 steps
unmapped = []
for c in cards:
    cn = card_node[c["card_id"]]
    if c.get("mes_equipment_id"):
        edge(cn, asset_of[c["mes_equipment_id"]], "SCOPED_TO",
basis="card.mes_equipment_id")
    for k in c.get("conditions") or []:
        sig = k["signal"][:-6] if k["signal"].endswith("_state") else
k["signal"]
        if k["signal"].endswith("_state") and k["op"] == "==" and f"st:{sig}=
```

```
{k['value']}" in nodes:
            edge(cn, f"st:{sig}={k['value']}", "APPLIES_WHEN",
basis="card.conditions")
        elif sig == "breaker_trip" and k["value"] == 1:
            edge(cn, "st:breaker_trip=high", "APPLIES_WHEN",
basis="card.conditions")
        elif k["signal"] == "fluid_viscosity" and k["op"] == ">":
            edge(cn, "st:fluid_viscosity=high", "APPLIES_WHEN",
basis="card.conditions (>2000 cSt, above the MES range)")
        else:
            unmapped.append((c["card_id"], k))
    for s in ((c.get("type_payload") or {}).get("steps") or []):
        st = node(f"step:{c['card_id']}/{s['step_id']}", "Step", s["action"]
[:60], order=s["order"], action=s["action"],
                  preconditions=s.get("preconditions") or [],
expected_result=s.get("expected_result"),
                  stop_conditions=s.get("stop_conditions") or [],
escalation=s.get("escalation_target"))
        edge(cn, st, "HAS_STEP", order=s["order"])

# proposed links
for src, roles in CURATED.items():
    for role, ids in roles.items():
        for cid in ids:
            edge(src, card_node[cid], "ADDRESSED_BY", role=role,
review_status="proposed")
for pat, syms in PATTERN_SYMPTOM.items():
    for s in syms:
        edge(pat, s, "INDICATES", review_status="proposed")

# validation
ids = set(nodes)
bad = [e for e in edges if e["source"] not in ids or e["target"] not in ids]
assert not bad, bad[:5]
assert len({(e["source"], e["target"], e["type"], json.dumps(e["props"],
sort_keys=True)) for e in edges}) == len(edges)
for e in edges:
    if e["type"] == "ADDRESSED_BY" and e["props"]["role"] != "safety":   # a
safety card is never offered as an action
        card = next(c for c in cards if card_node[c["card_id"]] ==
e["target"])
        assert card["tacit_type"] != "T5", (e["source"], card["card_id"])
co = [e for e in edges if e["type"] == "CO_OCCURRENCE"]
```

```
assert co and all(not e["props"]["causal"] for e in co)

counts = lambda key: dict(sorted(__import__("collections").Counter(x["type"]
for x in key).items()))
out.mkdir(parents=True, exist_ok=True)
graph = {"schema_version": "shiftlink-onto-v0-proposal", "extends": "kg-demo-
v1",
         "nodes": sorted(nodes.values(), key=lambda n: n["id"]), "edges":
edges}
data = json.dumps(graph, ensure_ascii=False, indent=1)
(out / "graph_v0.json").write_text(data, encoding="utf-8")
report = {"graph_sha256": hashlib.sha256(data.encode()).hexdigest(),
"node_types": counts(nodes.values()),
          "edge_types": counts(edges), "nodes": len(nodes), "edges":
len(edges),
          "unmapped_card_conditions": [[c, k] for c, k in unmapped],
          "proposed_edges": sum(1 for e in edges if
e["props"].get("review_status") == "proposed")}
(out / "validation_v0.json").write_text(json.dumps(report,
ensure_ascii=False, indent=1), encoding="utf-8")
print(json.dumps(report, ensure_ascii=False, indent=1))
```

validation_v0.json


```
{"graph_sha256":
"b8f4e154f65fc6c9c506461ca0750a12e6f520cd2b3143053293a9ecd285e80a",
"node_types": {"AlarmType": 11, "Asset": 10, "AssetPart": 18, "CardVersion":
86, "CauseHypothesis": 30, "Check": 30, "DocumentVersion": 92,
"EquipmentType": 83, "FailureMode": 41, "Line": 1, "PropagationRule": 6,
"SignalKind": 48, "SignalPoint": 69, "SignalState": 95, "Step": 108,
"Symptom": 18, "SymptomPattern": 15, "WaitReason": 5}, "edge_types":
{"ABOUT": 172, "ADDRESSED_BY": 73, "APPLIES_TO": 15, "APPLIES_WHEN": 15,
"CITES": 156, "COMMON_MODE": 1, "CO_OCCURRENCE": 1, "DEPENDENT_STATE": 13,
"DESCRIBES_SYMPTOM": 20, "DRIVE": 4, "HAS_FAILURE_MODE": 41, "HAS_PART": 18,
"HAS_SIGNAL": 69, "HAS_STEP": 108, "HYDRAULIC_SUPPLY": 4, "INDICATES": 7,
"INSTANCE_OF": 79, "INTERLOCK": 2, "MATERIAL_FLOW": 4, "PART_OF": 76,
"PNEUMATIC_SUPPLY": 1, "POWER_SUPPLY": 3, "PROVIDER_STATE": 8,
"RECOMMENDS_CHECK": 30, "REPORTS": 15, "REQUIRES": 32, "REQUIRES_NORMAL": 17,
"SCOPED_TO": 44, "STATE_OF": 95, "SUGGESTS": 30}, "nodes": 766, "edges":
1153, "unmapped_card_conditions": [["K-1029", {"signal":
"installation_outdoor_or_humid", "op": "==", "value": true, "unit": null}],
["K-1205", {"signal": "cv_belt_tension_state", "op": "==", "value": "normal",
"unit": null}]], "proposed_edges": 80}
```

부록 C. situation.py

```
"""Situation resolver prototype (10/22): MES snapshot -> evidence ->
situation via the proposed graph -> alert text.

Reads what a real MES would give: measurements of the last 3 frames, the MES
screen's pattern records
(engine.symptom_diagnostics), active alarm codes, operating states / wait
reasons. Never reads scenario ids, fault
truth or labels. Relations come from graph_v0.json (build_kg.py).

  python -X utf8 situation.py REPO GRAPH [--demo
pdp_current|pdp_current_gr_load|hydraulic_fault|gearbox_overheat]
"""
import argparse
import json
import sys
from collections import defaultdict
from datetime import datetime, timezone
from pathlib import Path

STATE_WORD = {"low": "낮음", "high": "높음", "zero": "0"}


class Graph:
    def __init__(self, path):
        g = json.loads(Path(path).read_text(encoding="utf-8"))
        self.n = {x["id"]: x for x in g["nodes"]}
        self.out, self.inc = defaultdict(list), defaultdict(list)
        for e in g["edges"]:
            self.out[(e["source"], e["type"])].append((e["target"],
e["props"]))
            self.inc[(e["target"], e["type"])].append((e["source"],
e["props"]))
        self.asset = {x["props"]["equipment_id"]: x["id"] for x in g["nodes"]
if x["type"] == "Asset"}
        self.eq_of = {v: k for k, v in self.asset.items()}
        self.deps = [(e["source"], e["target"], e["type"].lower(),
e["props"]) for e in g["edges"]
                     if e["props"].get("causal") and e["source"] in
self.eq_of and e["target"] in self.eq_of]
        self.rules = {x["id"]: x for x in g["nodes"] if x["type"] ==
"PropagationRule"}
```

```
    def label(self, i):
        return self.n[i]["label"] if i in self.n else i.split(":", 1)[1]


def evidence(engine):
    """Per equipment: sustained out-of-range states, screen patterns
(candidate only), alarms, operating state."""
    frames = list(engine._symptom_frames)[-3:]
    snap = engine.snapshot
    spec = {e.equipment_id: {s.signal: s for s in e.signals} for e in
engine.config.equipment}
    ev = defaultdict(lambda: {"states": {}, "patterns": [], "alarms": [],
"op": None, "wait": None})

    running = {e.equipment_id: e.operating_state == "running" for e in
snap.equipment}

    def state(eq, m):
        s = spec[eq].get(m.signal)
        if not s or m.quality != "good" or s.normal_min is None:
            return None
        if not running.get(eq, True) and s.zero_when_stopped:   # a stopped
unit reads 0: context, not a symptom
            return None
        if s.unit == "bool":
            return "high" if m.value >= 1 and s.normal_max < 1 else None
        if m.signal == "bus_current" and m.value == 0:
            return "zero"
        return "low" if m.value < s.normal_min else "high" if m.value >
s.normal_max else None

    for m in snap.measurements:
        st = state(m.equipment_id, m)
        if st and all(any(x.signal == m.signal and x.equipment_id ==
m.equipment_id and state(x.equipment_id, x) == st
                          for x in f.measurements) for f in frames):
            ev[m.equipment_id]["states"][m.signal] = (st, m.value, m.unit,
spec[m.equipment_id][m.signal])
    for d in engine.symptom_diagnostics():
        if d["status"] == "candidate":
            ev[d["equipment_id"]]["patterns"].append(d["pattern_id"])
    for a in snap.active_alarms:
```

```
        ev[a.equipment_id]["alarms"].append(a.code)
    for e in snap.equipment:
        if e.operating_state in ("stopped", "waiting"):
            ev[e.equipment_id]["op"], ev[e.equipment_id]["wait"] =
e.operating_state, e.wait_reason
    return ev


def own(x):
    return bool(x["states"] or x["patterns"] or x["alarms"])


def resolve(G, ev, code):
    situations = []
    for eq, x in ev.items():
        if not own(x):
            continue
        a = G.asset[eq]
        st_ids = {f"st:{s}={v[0]}" for s, v in x["states"].items()}
        pats = [f"pat:{p}" for p in x["patterns"]]
        sit = {"eq": eq, "code": code[eq], "x": x, "patterns": pats,
"upstream": [], "downstream": [], "affected": [],
               "cards": {}, "causes": [], "checks": []}
        for p in pats:
            sit["causes"] += [G.label(c) for c, _ in sorted(G.out[(p,
"SUGGESTS")], key=lambda t: t[1]["rank"])]
            sit["checks"] += [G.label(c) for c, _ in sorted(G.out[(p,
"RECOMMENDS_CHECK")], key=lambda t: t[1]["order"])]
        # relations: who could have caused this, whom it affects
        for src, dst, rel, props in G.deps:
            if dst == a:   # upstream cause only when a 'down' rule of this
relation matches the provider's state
                u = ev.get(G.eq_of[src])
                if u:
                    u_states = {f"st:{s_}={v[0]}" for s_, v in
u["states"].items()}
                    u_states |= {t for c in u["alarms"] for t, _ in
G.out[(f"alarm:{c}", "REPORTS")]}
                    if any(r["props"]["relation_type"] == rel and r["props"]
["direction"] == "down"
                           and {t for t, _ in G.out[(rid, "PROVIDER_STATE")]}
& u_states for rid, r in G.rules.items()):
                        sit["upstream"].append((code[G.eq_of[src]], rel))
```

```
            if src == a:
                d = ev.get(G.eq_of[dst])
                if d and d["wait"] and G.n.get(f"wait:{d['wait']}",
{}).get("props", {}).get("relation_type") == rel:
                    sit["affected"].append((code[G.eq_of[dst]], d["wait"]))
                for rid, rule in G.rules.items():   # a dependent's extra
demand seen in this provider's signal
                    if rule["props"]["relation_type"] != rel or rule["props"]
["direction"] != "up":
                        continue
                    if not {t for t, _ in G.out[(rid, "PROVIDER_STATE")]} &
st_ids:
                        continue
                    dep_states = {f"st:{s}={v[0]}" for s, v in (d or
{"states": {}})["states"].items()}
                    hit = {t for t, _ in G.out[(rid, "DEPENDENT_STATE")]} &
dep_states
                    sit["downstream"].append((code[G.eq_of[dst]], rel,
sorted(hit)))
        # knowledge: proposed links from patterns, alarms and states, and
exact card conditions
        srcs = pats + [f"alarm:{c}" for c in x["alarms"]] + sorted(st_ids)
        for s in srcs:
            for card, props in G.out[(s, "ADDRESSED_BY")]:
                sit["cards"].setdefault(card, props["role"])
        for st in st_ids:
            for card, _ in G.inc[(st, "APPLIES_WHEN")]:
                need = {t for t, _ in G.out[(card, "APPLIES_WHEN")]}
                scope = {t for t, _ in G.out[(card, "SCOPED_TO")]}
                if need <= st_ids and (not scope or a in scope):
                    sit["cards"][card] = "exact" if sit["cards"].get(card) in
(None, "partial") else sit["cards"][card]
        situations.append(sit)
    return situations


def render(G, sit):
    x, lines = sit["x"], []
    head = " · ".join(G.label(p) for p in sit["patterns"]) or ", ".join(
        f"{v[3].name.split(' (')[0]} {STATE_WORD[v[0]]}" for v in
x["states"].values()) or "경보"
    lines.append(f"[{sit['code']}] {head}  (자동 판정 · 확정 아님)")
    for s, (st, val, unit, sp) in x["states"].items():
```

```
        lines.append(f"  - {sp.name.split(' (')[0]} {val:g} {unit} (기준
{sp.normal_min:g}–{sp.normal_max:g}) {STATE_WORD[st]}")
    if x["alarms"]:
        lines.append("  - MES 경보: " + ", ".join(G.label(f"alarm:{c}") for c
in x["alarms"]))
    if sit["causes"]:
        lines.append("  원인 후보: " + " /
".join(dict.fromkeys(sit["causes"])))
    for u, rel in sit["upstream"]:
        lines.append(f"  관계: 상류 {u}({rel})에도 이상 → 상류 원인 가능")
    for d, rel, hit in sit["downstream"]:
        lines.append(f"  관계: {d}({rel}) " + (f"부하 신호 {', '.join(G.label(h)
for h in hit)} → 하류 부하 증가 후보"
                                              if hit else "부하 신호 정상 → 하
류 부하 증가 근거 없음"))
    for d, why in sit["affected"]:
        lines.append(f"  영향: {d} 대기({why})")
    if sit["checks"]:
        lines.append("  확인할 것: " + " /
".join(dict.fromkeys(sit["checks"])))
    act = [(c, r) for c, r in sit["cards"].items() if r in ("primary",
"exact")]
    other = [(c, r) for c, r in sit["cards"].items() if r in ("partial",
"relation")]
    safety = [c for c, r in sit["cards"].items() if r == "safety"]
    for c, r in act:
        lines.append(f"  추천 조치 [{G.n[c]['props']['card_id']}]
{G.label(c)}")
        for step, _ in sorted(G.out[(c, "HAS_STEP")], key=lambda t: t[1]
["order"])[:3]:
            p = G.n[step]["props"]
            lines.append(f"     {p['order']}. {p['action'][:70]}" + (f"  · 멈
춤: {p['stop_conditions'][0][:40]}" if p["stop_conditions"] else ""))
    for c, r in other:
        lines.append(f"  참고 [{G.n[c]['props']['card_id']}] {G.label(c)}
({'현장 관측 필요' if r == 'partial' else '연결 설비'})")
    for c in safety:
        lines.append(f"  안전 [{G.n[c]['props']['card_id']}] {G.label(c)}")
    if not act:
        lines.append("  지식 공백: 이 상황에 직접 쓰는 카드가 없습니다 → 조치 후 근무일지
기록 시 카드 후보로 검토")
    return "\n".join(lines)
```

```
def demo(repo, which):
    sys.path.insert(0, str(repo))
    from shiftlink.mes.configuration import from_catalog
    from shiftlink.mes.contracts import Run
    from shiftlink.mes.engine import MesEngine
    from shiftlink.mes.random_factory import RandomFactoryEngine
    from shiftlink.mes.scenarios.priority import expand
    config = expand(from_catalog(json.loads((repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))))
    run = Run.create(5, datetime(2026, 10, 22, tzinfo=timezone.utc),
config.config_id)
    if which in ("hydraulic_fault", "gearbox_overheat", "pdp_current"):
        eng = MesEngine(run, config); eng.start(); eng.tick(4)
        eng.set_scenario("sensor_anomaly_EQ-0002_bus_current" if which ==
"pdp_current" else which); eng.tick(4)
        return eng
    eng = RandomFactoryEngine(run, config, run_nonce="demo",
faults_enabled=False); eng.start(); eng.tick(4)
    for eq, pid in (("EQ-0002", "pdp_current"), ("EQ-0004", "gr_load")):   #
inject two random-factory faults by hand
        eng.scheduler.enabled = True
        opts = dict((p.pattern_id, (p, s)) for p, s in
eng.scheduler.options[eq])
        eng.scheduler.options[eq] = [opts[pid]]
        f = eng.scheduler._new(eq, eng.snapshot.sequence + 1);
f.update(mode="abrupt", severity=1.0, end=f["start"] + 30)
        eng.scheduler.active.append(f)
    eng.scheduler.enabled = False
    eng.tick(5)
    return eng


if __name__ == "__main__":
    ap = argparse.ArgumentParser(); ap.add_argument("repo", type=Path);
ap.add_argument("graph", type=Path)
    ap.add_argument("--demo", default="pdp_current"); a = ap.parse_args()
    G = Graph(a.graph)
    eng = demo(a.repo, a.demo)
    code = {e.equipment_id: e.code for e in eng.config.equipment}
    for sit in resolve(G, evidence(eng), code):
        print(render(G, sit), "\n")
```

eval_scenarios.py

```
"""Baseline (10/22) of the resolver on MES scenarios outside the random-
factory pattern table: 14 named + 69 single-signal
sensor anomalies. Truth (cause equipment from the scenario spec) is read only
for scoring.
 root  = a situation on the cause equipment            V0 = MES screen
patterns only (current engine)
 blame = situations on other equipment (wrong blame)    card = cause
situation has a primary/exact card
  python -X utf8 eval_scenarios.py REPO GRAPH"""
import json, sys
from collections import Counter
from datetime import datetime, timezone
from pathlib import Path
repo, gpath = Path(sys.argv[1]), Path(sys.argv[2]); sys.path.insert(0,
str(repo))
from situation import Graph, evidence, resolve
from shiftlink.mes.configuration import from_catalog
from shiftlink.mes.contracts import Run
from shiftlink.mes.engine import MesEngine
from shiftlink.mes.scenarios.priority import expand
config = expand(from_catalog(json.loads((repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))))
code = {e.equipment_id: e.code for e in config.equipment}
G = Graph(gpath); res = Counter(); rows = []
for spec in config.scenarios:
    sid = spec.scenario_id
    if sid == "normal" or sid.startswith("symptom_") or sid in
("scrap_discharge", "coil_quality_hold", "hpu_accumulator_precharge"):
        continue   # not faults: a production step, a quality hold, a
maintenance measurement
    eng = MesEngine(Run.create(9, datetime(2026, 10, 22,
tzinfo=timezone.utc), config.config_id), config)
    eng.start(); eng.tick(4); eng.set_scenario(sid); eng.tick(4)
    cause = eng._cause_equipment_id(spec)
    sits = resolve(G, evidence(eng), code)
    v0 = any(d["equipment_id"] == cause and d["status"] == "candidate" for d
in eng.symptom_diagnostics())
    mine = [s for s in sits if s["eq"] == cause]
    card = bool(mine and any(r in ("primary", "exact") for r in mine[0]
["cards"].values()))
```

```
    kind = "sensor" if sid.startswith("sensor_anomaly_") else "named"
    res[(kind, "n")] += 1; res[(kind, "v0")] += v0; res[(kind, "root")] +=
bool(mine)
    res[(kind, "blame")] += sum(1 for s in sits if s["eq"] != cause);
res[(kind, "card")] += card
    rows.append({"scenario": sid, "cause": code[cause], "v0": v0, "root":
bool(mine), "card": card,
                 "others": [s["code"] for s in sits if s["eq"] != cause],
                 "affected": [a for s in mine for a in s["affected"]]})
for kind in ("named", "sensor"):
    n = res[(kind, "n")]
    print(f"{kind:6s} n={n}  V0 screen root {res[(kind,'v0')]}/{n}  resolver
root {res[(kind,'root')]}/{n}  "
          f"wrong-blame situations {res[(kind,'blame')]}  primary/exact card
{res[(kind,'card')]}/{n}")
for r in rows:
    if not r["scenario"].startswith("sensor_") or r["others"] or not
r["root"]:
        print(" ", r)
Path("out/eval_scenarios.json").write_text(json.dumps(rows,
ensure_ascii=False, indent=1), encoding="utf-8")
```

부록 D. probe_screen.py · probe_named.py

```
"""Cheap probe (10/22): does MES screen_symptoms recognise the faults the
random factory injects?
Event = one injected fault episode (truth id). Detected = a diagnostic record
on the same equipment with the same
pattern_id while the fault is active (+2 ticks). 'candidate' = sustained 3
frames (alert-grade), 'observing' = 1-2.
The screen reads measurements only; the truth is read only for scoring.
  python -X utf8 probe_screen.py REPO [--seeds 1 2 3 4] [--ticks 900] [--
hazard .015]"""
import argparse, collections, json, sys
from datetime import datetime, timezone
from pathlib import Path
ap = argparse.ArgumentParser(); ap.add_argument("repo", type=Path)
ap.add_argument("--seeds", type=int, nargs="+", default=[1, 2, 3, 4]);
ap.add_argument("--ticks", type=int, default=900)
ap.add_argument("--hazard", type=float, default=.015); a = ap.parse_args()
sys.path.insert(0, str(a.repo))
from shiftlink.mes.configuration import from_catalog
```

```
from shiftlink.mes.contracts import Run
from shiftlink.mes.random_factory import RandomFactoryEngine
from shiftlink.mes.scenarios.priority import expand
config = expand(from_catalog(json.loads((a.repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))))
code = {e.equipment_id: e.code for e in config.equipment}
ev = {}; fp = collections.Counter(); wrong = collections.Counter();
frames_with_fault = 0; normal_alert_frames = 0
for seed in a.seeds:
    eng = RandomFactoryEngine(Run.create(seed, datetime(2026, 10, 22,
tzinfo=timezone.utc), config.config_id), config,
                              run_nonce=f"probe{seed}", faults_enabled=True,
hazard=a.hazard)
    eng.start()
    for _ in range(a.ticks):
        snap = eng.tick()
        t = eng.truth[-1] if eng.truth else {"faults": []}
        act = {(f["equipment_id"]): f for f in t["faults"]}
        for f in t["faults"]:
            e = ev.setdefault((seed, f["id"]), {"eq": f["equipment_id"],
"pattern": f["pattern_id"], "mode": f["mode"],
                                                "sev": round(f["severity"],
2), "start": f["start"], "end": f["end"],
                                                "first_obs": None,
"first_cand": None})
        for d in eng.symptom_diagnostics():
            f = act.get(d["equipment_id"])
            if f and f["pattern_id"] == d["pattern_id"]:
                e = ev[(seed, f["id"])]; k = "first_cand" if d["status"] ==
"candidate" else "first_obs"
                if e[k] is None: e[k] = snap.sequence - f["start"]
                if d["status"] == "candidate" and e["first_obs"] is None:
e["first_obs"] = e["first_cand"]
            elif d["status"] == "candidate":
                (wrong if f else fp)[(code[d["equipment_id"]],
d["pattern_id"], f["pattern_id"] if f else None)] += 1
n = len(ev); cand = [e for e in ev.values() if e["first_cand"] is not None];
obs = [e for e in ev.values() if e["first_obs"] is not None]
print(f"episodes {n} | detected(candidate) {len(cand)}
({len(cand)/max(1,n):.0%}) | any(observing) {len(obs)}
({len(obs)/max(1,n):.0%})")
lat = sorted(e["first_cand"] for e in cand); print("candidate latency ticks
```

```
p50/p95", lat[len(lat)//2] if lat else None, lat[int(len(lat)*.95)] if lat
else None)
for k in ("mode", "pattern"):
    c = collections.defaultdict(lambda: [0, 0])
    for e in ev.values(): c[e[k]][0] += 1; c[e[k]][1] += e["first_cand"] is
not None
    print(k, {kk: f"{v[1]}/{v[0]}" for kk, v in sorted(c.items())})
print("candidate records on fault-free equipment (frames):",
sum(fp.values()), fp.most_common(5))
print("candidate records with a different pattern on a faulty unit
(frames):", sum(wrong.values()), wrong.most_common(5))
```

```
"""Cheap probe (10/22): which screen_symptoms patterns fire for the MES
scenarios that are NOT generated from the pattern
table (14 named + 69 single-signal sensor anomalies)? For each scenario:
fresh engine, 4 normal ticks, set_scenario,
6 ticks; report diagnostics (pattern, status), alarms, and which equipment
changed operating state (propagation).
  python -X utf8 probe_named.py REPO"""
import collections, json, sys
from datetime import datetime, timezone
from pathlib import Path
repo = Path(sys.argv[1]); sys.path.insert(0, str(repo))
from shiftlink.mes.configuration import from_catalog
from shiftlink.mes.contracts import Run
from shiftlink.mes.engine import MesEngine
from shiftlink.mes.scenarios.priority import expand
config = expand(from_catalog(json.loads((repo /
"docs/data/reference/00_plant_and_relations.json").read_text(encoding="utf-
8"))))
code = {e.equipment_id: e.code for e in config.equipment}
rows = []
for spec in config.scenarios:
    if spec.scenario_id == "normal" or
spec.scenario_id.startswith("symptom_"):
        continue
    eng = MesEngine(Run.create(7, datetime(2026, 10, 22,
tzinfo=timezone.utc), config.config_id), config)
    eng.start(); eng.tick(4)
    try:
        eng.set_scenario(spec.scenario_id)
    except ValueError as err:
        rows.append((spec.scenario_id, "set_scenario error", str(err)[:60]));
```

```
continue
    pats = collections.Counter(); alarms = set(); states = set()
    for _ in range(6):
        snap = eng.tick()
        for d in eng.symptom_diagnostics():
            pats[(code[d["equipment_id"]], d["pattern_id"], d["status"])] +=
1
        alarms |= {(code.get(al.equipment_id, al.equipment_id), al.code) for
al in snap.active_alarms}
        states |= {(code[e.equipment_id], e.operating_state, e.wait_reason)
for e in snap.equipment if e.operating_state != "running"}
    rows.append((spec.scenario_id, snap.line_mode, sorted({k[:2] for k, v in
pats.items()}), sorted(alarms), sorted(states)))
named = [r for r in rows if not r[0].startswith("sensor_")]
for r in named: print(r)
sens = [r for r in rows if r[0].startswith("sensor_")]
hit = [r for r in sens if len(r) > 2 and r[2] and r[2] != "set_scenario
error" and isinstance(r[2], list) and r[2]]
print(f"sensor anomalies: {len(sens)} scenarios; any pattern fired:
{len(hit)}")
for r in hit[:12]: print("  ", r[0], r[2])
```

부록 E. PLAN.md

5절 실험 계획의 전문이다. 5절에 없는 내용만 옮긴다.

비교 팔(arm)

- H1 상황 인식: V0 MES 패턴만(현재) / V1 +신호 상태(운전 문맥) / V2 +경보 / V3 +대기 사유
- H2 원인 위치: R0 관계 없음 / R1 관계 + 규칙(대기 사유 제외) / R2 전체
- H3 지식 연결: K0 카드 조건만(PDA `rankCards`) / K1 그래프 / K2 S30h 검색(상황 문장 자동 생성) / K3 그래프 + 공백일 때만 검색
- H4 알림 품질: K1·K3·K2 알림 문장을 블라인드로 섞어 1–5점 평가
- H5 장비: C5 단독 / C5 + 판정기

데이터

	출처	용도
D1	무작위 고장(현재 생성기) dev seed 1–8 / test 101–108, 900틱, hazard .015	위생 점검만(판정기와 같은 표)

	출처	용도
D2	D1의 약화·간헐 변형(강도 .2–.5)	H1
D3	이름 붙은 고장 11 + 신호 이상 69, 각 3 seed, 경보를 가린 변형 포함	H1·H2·H3
D4	정상 운전 8 run × 900틱	오알림
D5	동시·전파 고장 스크립트(PDP+GR 부하, HPU 압력+RT 클램프, CV 정체+RT 속도)	H2
D6	(가능하면) 팀원 v7 coherent 생성기	H2 최강 근거

실패 분류

코드	분류
F1	센서로 관측 불가(소음·거품·색)
F2	패턴·상태 미정의
F3	운전 문맥 오판(v0에서 수정)
F4	관계 규칙 누락·오적용(v0에서 수정)
F5	카드 링크 누락
F6	진짜 지식 공백
F7	정답 모호
F8	알림 폭주·중복
F9	안전 게이트 위반

범위 밖(다음 단계 후보): EWMA 조기 경보를 증거로 추가, 그래프로 RAG 질의 확장(GraphRAG), 지식 공백 → 근무일지 → 카드 게시 자동 연결, GNN은 현장 데이터가 쌓인 뒤.